# Chapter 4: Reasoning models

Standard Large Language Models are astonishingly good at producing fluent, coherent text. But their architecture ensures that they hallucinate and make mistakes. In particular, they struggle with problems that require sustained logical reasoning, multi-step mathematics, or careful verification of intermediate steps. 'Reasoning models' attempt to address this limitation by building in a deliberate 'thinking' process before answering.

## 4.1 Chain of Thought

Consider this problem: A farmer has 15 cows. He sells 7 of them. Then, his remaining cows have calves, and the herd doubles in size. Finally, a storm causes him to lose 3 cows. How many cows does the farmer have at the end? The correct answer is 13.

If you ask GPT-4o (an old model), this is how it goes:

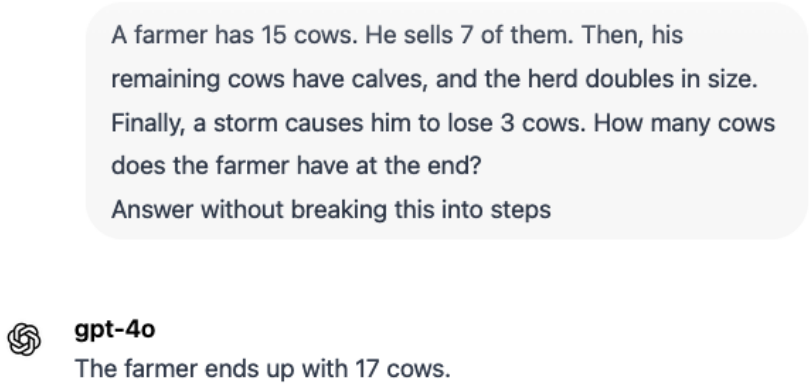



**Figure 4.1.** GPT-4o answers the farmer problem directly, without being asked to reason step by step.

This is the wrong answer. This is not a very hard question, so how could it get it wrong? Because the LLM is not doing any math. All it's doing is predicting the next word and it predicts 17, not based on any calculation, just on the frequency with which the number 17 appears in sentences like this.

This doesn't mean that LLMs can't do any math. If you ask an LLM what 8 + 8 is, it will almost always get the correct answer (16) because it's been trained on math tables and this sum appears there, so its predictive algorithm produces the right answer.

But if you ask it a question that never appeared in the training data and that requires multiple steps, traditional LLMs fail more often than they succeed.

Soon after the first LLMs were released, however, people realized that simply telling the model to break the problem down into steps led to an increase in the ability of the models to do these problems:

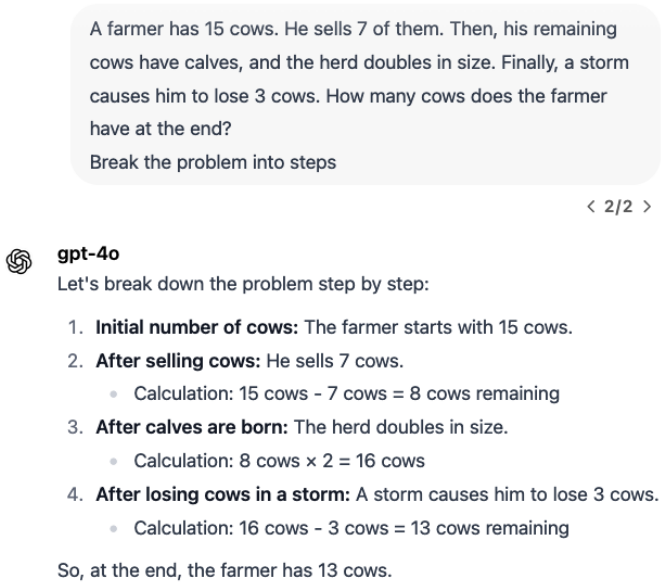



**Figure 4.2.** The same question, now prompted to break the problem into steps.

This last example shows what's known as "chain of thought" (CoT), a prompting technique where the model is told to "think step by step," and the model generates a sequence of reasoning steps. Any LLM can do this — it's not a special architecture, just a way of prompting the model to produce a more structured, step-by-step solution instead of jumping straight to an answer.

In this example, the LLM is still doing what it always does — predicting the next word. But by asking the LLM to first break the problem into steps, the LLM's next-word prediction leads it to break the problem into a series of simple math problems that it can sequentially solve. In the example above, the first calculation is 15 minus 7, which is can accurately determine is 8. Then is calculates 8 times 2, which is again easy for the LLM. Finally, it calculates 16 minus 3, which is again easy for the LLM.

Let me repeat: CoT is generated word-by-word, just like normal text from an LLM. There's no real thinking. It still doesn't understand what it's doing.

## 4.2 Reasoning models

Based on the observation that asking models think step-by-step led to superior performance, the major AI labs started producing "reasoning models". A reasoning model is an LLM that has been trained to do the 'think step by step' part on its own (without you asking), at length, and before it commits to an answer.

When a reasoning model receives a question, it first generates a long stretch of text in which it breaks down the problem — its 'chain of thought,' in the term from the previous section, though 'scratchpad' is a better picture of what it is — and only then writes the reply you see.

Think of the scratchpad as the place where the model jots down intermediate calculations, notes where an approach seems to lead, backtracks when something does not add up, and settles on a conclusion. OpenAI's o1, released in late 2024, was the first widely used reasoning model; DeepSeek's R1 followed in January 2025, and by 2026 the ability to think before answering is built into essentially every major model: OpenAI, Claude, Gemini, DeepSeek, Qwen.

Here is what the scratchpad looks like for the farmer problem from earlier in this chapter. This is a condensed, illustrative version of the monologue a reasoning model produces before it answers; real ones can be longer and messier:

> *Okay, let me work through this. Start with 15 cows. He sells 7, so 15 − 7 = 8 cows left. Then the remaining cows have calves and the herd doubles in size, so 8 × 2 = 16. Then a storm and he loses 3, so 16 − 3 = 13.*
> *Wait — let me re-read the question. 'His remaining cows have calves, and the herd doubles.' Does the herd double from 8, or is it the original 15 that doubles? It says 'remaining,' so it is the 8 that doubles. 16. Good.*
> *One more possible catch: the question asks how many cows, and half the herd are now calves. Do calves count? The problem says the 'herd doubles in size'*

*and then that he loses '3 cows,' so it is treating every animal in the herd as a cow. I will go with that reading, but I could mention it.*
*Let me verify by running it backwards: 13 + 3 = 16, 16 ÷ 2 = 8, 8 + 7 = 15. That matches the starting number.*
*Answer: 13.*

Three things in this monologue are worth noticing. First, the model re-reads the question after it already has an answer and asks whether it interpreted the words correctly. Second, it notices an ambiguity (are calves cows?) and decides how to handle it rather than plowing ahead. Third, it checks its result — here, by running the calculation in reverse and confirming that it lands back on the number the problem started with.

If a check fails, the model goes back and tries again. Recall that the old model at the start of this chapter simply produced '17' and stopped; it had no step in which it could notice that 17 was wrong. A reasoning model gives itself that step, and for anything with several stages — long multiplication, a coding task, a multi-part logic puzzle — the checking is where most of the improvement comes from.

Real scratchpads can be longer and messier than the example above. They are full of 'Wait,' 'Hmm,' 'Actually, let me reconsider,' and 'Let me double-check that.' Models abandon an approach halfway through and start over, try a second method to see whether it gives the same answer, and sometimes talk themselves in circles for thousands of words before settling.

In the paper describing DeepSeek's R1, the researchers reproduced a moment, midway through an algebra problem, where the model interrupted itself with 'Wait, wait. Wait. That's an aha moment I can flag here' and restarted the solution from the beginning. Nobody wrote that behavior into the model; as the next section explains, it emerged from training.

It is also important to keep in mind that none of this changes what the LLM fundamentally is. The scratchpad is generated one token at a time, by the same next-word prediction that LLMs use. What has changed is that the model now spends many of those tokens working

on the problem instead of committing to an answer immediately — and, as a rule, the more tokens it spends thinking, the better it does on hard problems.

## 4.3 How models learn to reason

What makes reasoning models different from simply asking a standard LLM to 'think step by step' is how they are trained. Reasoning models are specifically trained to maximize the effectiveness of their step-by-step thinking.

The key training innovation is the use of reinforcement learning (RL) applied specifically to verifiable reasoning tasks. For problems like mathematics and coding, there is a ground truth: an answer is either correct or it is not. The model can be trained by generating many candidate reasoning paths, checking which ones lead to correct answers, and reinforcing the strategies that worked. Unlike RLHF — which relies on human raters making subjective judgments — this process can be automated, since correctness can be verified programmatically.

DeepSeek published its recipe in January 2025, so we know what this looks like in practice. Start with an ordinary LLM. Give it a large collection of math and programming problems whose answers can be checked automatically — a math answer is compared against an answer key, a program is run against test cases. Have the model attempt each problem many times, writing out its reasoning inside a designated 'thinking' section before giving a final answer. Reward the attempts that end in the correct answer, penalize the ones that do not, and nudge the model's parameters toward whatever it did in the successful attempts.

Then repeat, for thousands of rounds. Nobody tells the model how to reason — the reward is for the final answer only. But over the course of training, the model's thinking sections grow from a few sentences to thousands of words, and they spontaneously fill up with the behaviors in the example above: re-reading the question, checking the arithmetic, trying a second approach, backing up when something does not add up. Those behaviors survive because, on average, the attempts that included them got the answer right more often.

This does not mean reasoning models are reasoning in the way humans reason. They are still susceptible to certain types of errors, particularly in novel situations that differ

significantly from their training distribution. But the improvement over standard LLMs on structured reasoning tasks is substantial and represents a genuine advance.

### 4.4 Who gets to see the scratchpad?

In ChatGPT, Claude, and Gemini, users see a summary of the reasoning process displayed as 'thinking' steps before the final response appears. However, the raw chain-of-thought is not shown — the company generates a human-readable summary of it rather than exposing the actual internal tokens.

This decision was made for a combination of reasons: user experience (raw reasoning can be difficult to read), competitive advantage (the reasoning process is the product, and a visible one can be copied), and safety considerations. OpenAI explicitly prohibits users from attempting to extract the hidden chain-of-thought, and monitors for such attempts.

DeepSeek's R1 model, released in January 2025, makes its chain-of-thought visible. This transparency allows researchers and users to inspect the reasoning process and identify where the model goes wrong — a significant advantage for debugging and building trust.

### 4.5 Trade-offs: how much thinking do you need?

When reasoning models first appeared, whether to use a reasoning model was a decision: OpenAI's reasoning o1 and the 'standard' GPT-4o were separate products, and you picked one or the other. By 2026, that distinction has mostly disappeared. Now, most models are reasoning models so the question for a user is no longer 'which model?' but 'how much thinking is this question worth?'

The 'thinking tokens' consumed during the reasoning process also takes time to generate. A question that a model answers in two seconds with thinking turned off might take thirty seconds, or several minutes, with it turned on, as the model works through its internal chain-of-thought. This is a deliberate trade-off: accuracy over speed.

Because someone pays for every thinking token, a hard problem that burns through several thousand of them can cost five to twenty times as much as the same question answered with thinking off (which uses no thinking tokens). There is also such a thing as

overthinking: on easy questions, a model given a large thinking budget will sometimes second-guess a correct first answer into a wrong one, so more thinking is not automatically better.

For example, here is chatGPT's interface, where I have thinking set to medium.

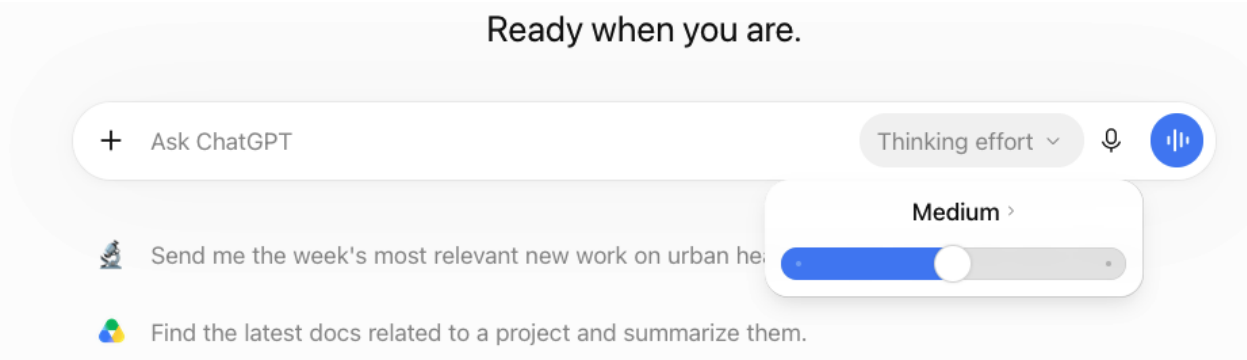



**Figure 4.3.** ChatGPT's interface with the reasoning effort set to Medium.

You should turn thinking up for: multi-step mathematical problems, complex coding tasks and debugging, logical puzzles and formal reasoning, tasks requiring careful verification of intermediate steps, and scientific or technical analysis where accuracy is more important than speed. For these use cases, the improvement from extra thinking can be dramatic.

Turn it down, or off, for: high-volume, low-stakes tasks where speed matters (drafting emails, summarizing documents), creative writing where constraint and deliberation can actually reduce quality, real-time conversational applications, and cost-sensitive deployments where the additional compute for reasoning is too expensive.

It is also important to maintain realistic expectations about what 'reasoning' means in this context. Reasoning models do not possess general intelligence or human-like understanding. They are still pattern-matching systems that have been trained to generate reasoning-like text. Therefore, they are still susceptible to hallucination.

## 4.6 Key takeaways

- A hidden 'chain-of-thought' scratchpad allows the model to work through intermediate steps and check its own result before delivering its final answer.

- This represents a genuine advance, but reasoning models remain statistical systems, not true logical reasoners.
- Reasoning models are trained using reinforcement learning on verifiable tasks (like math problems), rewarding reasoning paths that lead to correct answers. The checking and backtracking behaviors emerge from that reward rather than being programmed in.
- Extended thinking dramatically improves performance on math, coding, and logic, but it is slower, more expensive, and not universally better — and most current models let you choose how much thinking to use.